



Master 1 (M1)

***Département d'informatique
l'UFR-SEA/
UJKZ***

Bases de Données Avancées

***Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ***

Janvier 2023

<http://kisitokab.xyz>



- **Prérequis**
 - Principes des systèmes de base de données
- **Volume horaire**
 - 30 heures
 - 3 Crédits
- **Méthode d'évaluation**
 - 1 Devoir
 - 1 Projet



Justification du Cours/ module



Résultats de l'apprentissage



Contenu-sujets du Cours

- Introduction à la conception des BdD
- Introduction à la réalisation des BdD
- Introduction à l'administration des BdD
- Les BdD réparties
- Les BdD Objets
- Les extensions XML



Master 1 (M1)

*IBAM
UJKZ*

CHAPITRE 4

Gestion des transactions

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ*

Janvier 2021

<http://kisitokab.xyz>

Objectif

Gestion des transactions

Chapitre 12 - Objectifs

Le but de la restauration de base de données,
Le rôle et l'importance des transactions,
Les propriétés d'une transaction

Soutien des transactions

- **Transaction**

- Une action ou une suite d'actions demandée par un seul utilisateur ou programme d'application, qui lit ou met à jour le contenu de la base de données.
- Une transaction est une unité logique de travail sur la base de données. Ce peut être un programme tout entier, une partie de programme ou une seule commande (par exemple, une commande SQL du genre INSERT ou UPDATE) qui peut impliquer un nombre quelconque d'opérations sur la base de données. Dans le contexte de la base de données, l'exécution d'un programme d'application peut se voir comme une ou plusieurs transactions avec du calcul intermédiaire qui ne fait pas appel à la base de données.
- Pour illustrer le concept de transaction, examinons deux relations de l'instance de la base de données locative *Maisons de Rêve*

Exemples de transactions

```
lire(numPersonnel = x, salaire)
nouv_salaire = salaire * 1.1
écrire(numPersonnel = x, nouv_salaire)
```

a)

```
supprimer(numPersonnel = x)
pour tous les enregistrements de PropriétéALouer, numProp
début
    lire(numPropriété = numProp, numPersonnel)
    si (numPersonnel = x) alors
        numPersonnel = nouvNumPers
        écrire(numPropriété = numProp, numPersonnel)
    fin
fin
```

b)

Personnel	(<u>numPersonnel</u> , prénom, nom, fonction, genre, dateDeNaissance, salaire, numfilliale)
PropriétéALouer	(<u>numPropriété</u> , rue, ville, codePostal, type, pièces, location, numPropriétaire, numPersonnel, numFilliale)

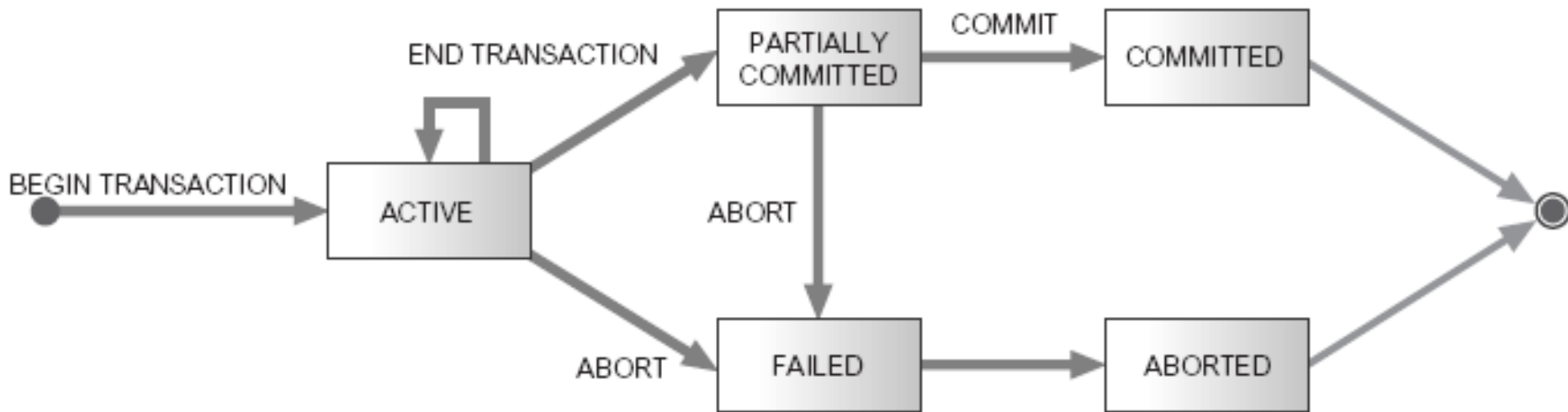
Soutien des transactions

- Une transaction doit toujours transformer la base de données d'un état cohérent en un autre état cohérent, même si nous sommes obligés d'accepter que la cohérence soit violée pendant la progression de la transaction.
- Une transaction peut donner deux résultats possibles. Si elle s'achève avec succès, la transaction est dite **validée** ou **confirmée** (*committed*) et la base de données atteint un nouvel état cohérent.
- Par contre, si la transaction ne s'exécute pas avec succès, la transaction est **avortée** (*aborted*). Dans ce cas, la base de données doit retrouver l'état cohérent qu'elle avait avant le début de la transaction. Une transaction qui subit cette réinitialisation est dite **annulée** (*rolled back*).

Soutien des transactions

- Une transaction validée ne peut plus être annulée.
- Si nous considérons après coup qu'une transaction validée était erronée, la seule manière de l'annuler est d'exécuter une **transaction de compensation** pour en inverser les effets.
- D'un autre côté, une transaction annulée peut être relancée plus tard et, si la cause de son annulation a disparu, peut s'exécuter cette fois avec succès et être validée..

Le diagramme de transitions d'états d'une transaction.



Propriétés des transactions

„ Les 4 propriétés de base:

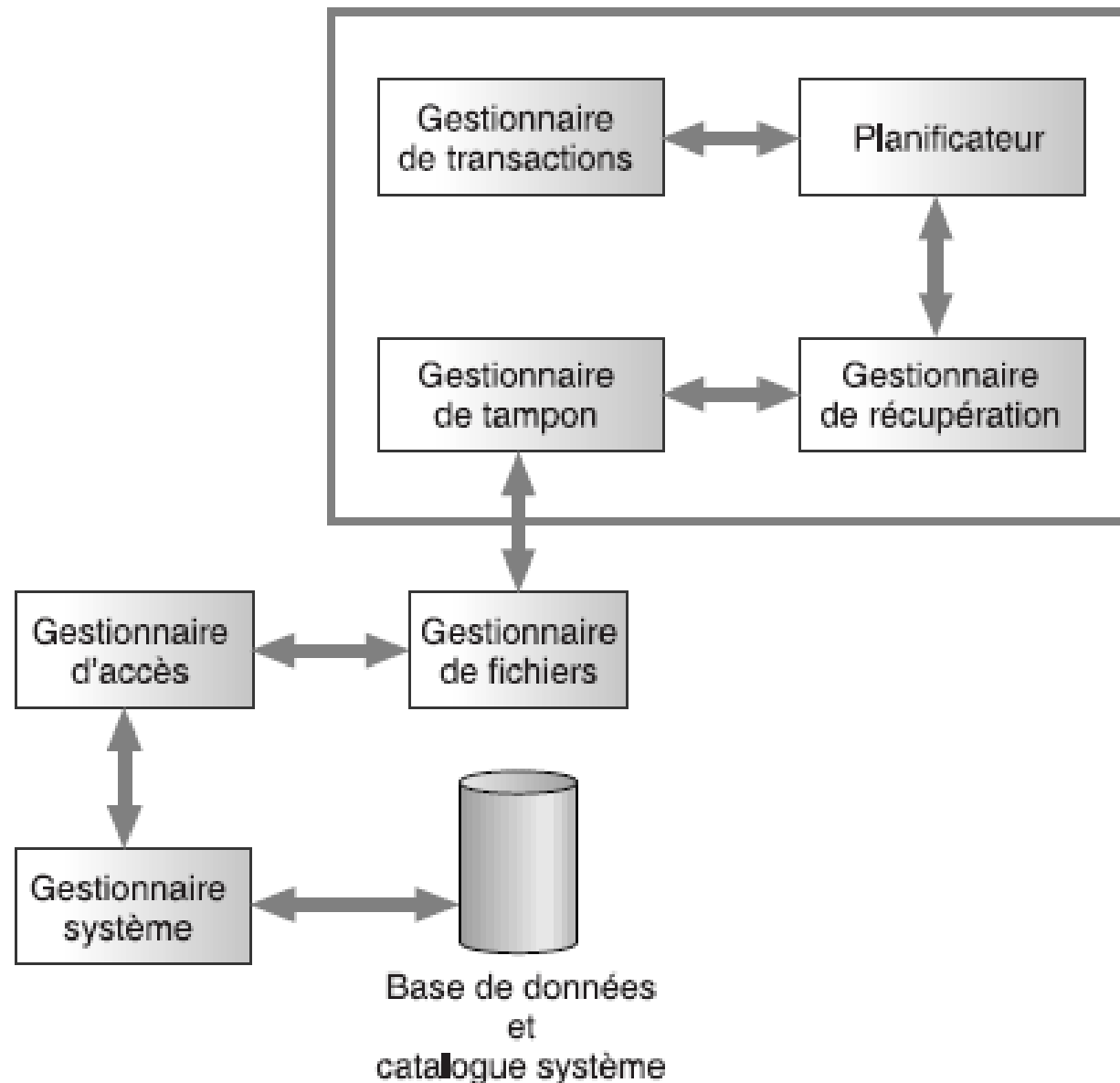
... Atomicité : 'tout ou rien',

... Cohérence : Doit transformer la BD d'un état cohérent à un autre,

... Isolation : Les effets partiels de transactions incomplètes ne peuvent pas être visible aux autres transactions,

... Durabilité : Les effets d'une transaction validée sont permanents et ne doivent pas être perdus par des défaillances subséquentes.

Sous-système de transaction d'un SGBD



Contrôle de concurrence

- „ Processus de gestion des opérations simultanées sur la BD sans qu'elles n'interfèrent entre elles,
- „ Préviend les incohérences lorsque deux ou plusieurs utilisateurs accèdent à la BD de façon simultanée et qu'au moins 1 est à modifier les données,
- „ Bien que deux transactions soit correctes en elles-mêmes, l'interfoliage des opérations peut produire un résultat incorrect,

3 exemples des problèmes potentiels de la concurrence:

- ... Problème de mise à jour perdue,
- ... Problème de dépendance non validée,
- ... Problème de l'analyse incohérente.

11

Problème de mise à jour perdue

- „ Une mise à jour complétée avec succès peut être écrasée par un autre utilisateur,
- „ T1 retire 10\$ d'un compte dont le solde est initialement de 100\$,
- „ T2 dépose 100\$ dans le même compte,
- „ À la fin, le solde final est de 90\$.

Problème de mise à jour perdue

Temps	T ₁	T ₂	solde _x
t ₁		début_transaction	100
t ₂	début_transaction	lire(solde _x)	100
t ₃	lire(solde _x)	solde _x = solde _x + 100	100
t ₄	solde _x = solde _x - 10	écrire(solde _x)	200
t ₅	écrire(solde _x)	validation	90
t ₆	validation		90

Capacité de sérialisation et de récupération

L'objectif du protocole de contrôle de la concurrence est de planifier les transactions de manière à éviter toute interférence entre elles et, donc, d'éviter les problèmes des types évoqués à la section précédente. Une solution évidente consiste à ne permettre l'exécution que d'une seule transaction à la fois : une transaction est *validée* avant d'autoriser la transaction suivante à *débuter*.

Or, la raison d'être du SGBD est d'atteindre un niveau optimal de simultanéité, de parallélisme dans le système, de sorte que les transactions puissent s'exécuter sans interférence, certes, mais surtout autant que possible en parallèle.

Par exemple, les transactions qui accèdent à des parties complètement distinctes d'une même base de données peuvent être planifiées ensemble, sans risquer d'interférence.

Capacité de sérialisation et de récupération

Planification (*schedule*)

Une séquence d'opérations d'un ensemble de transactions concurrentes qui préserve l'ordre des opérations dans chacune des transactions.

Planification sérielle

Une planification où les opérations de chaque transaction sont exécutées de manière consécutive, sans aucune opération interfoliée d'autres transactions.

Capacité de sérialisation et de récupération

Planification non sérielle

Une planification où les opérations d'un ensemble de transactions sont exécutées de manière interfoliée.

Dans le contexte de la capacité de sérialisation, l'ordre des opérations de lecture et d'écriture importe :

- Si deux transactions ne font que lire des données, elles n'entrent pas en conflit et leur ordre est sans importance.
- Si deux transactions soit lisent, soit écrivent complètement des données différentes, elles n'entrent pas en conflit et leur ordre est sans importance.
- Si une transaction écrit dans des données et si une autre transaction lit ou écrit dans ces mêmes données, alors l'ordre de leur exécution importe.

Test de conflits de la capacité de sérialisation

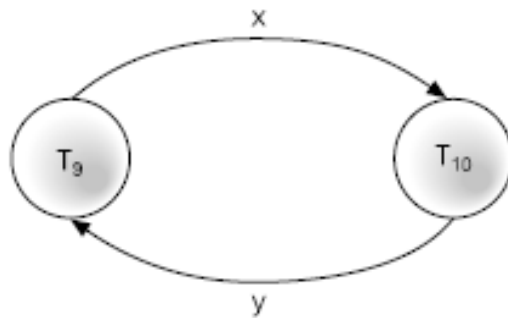
Sous la **règle d'écriture contrainte** (c'est-à-dire qu'une transaction modifie une donnée en fonction de son ancienne valeur, lue en premier lieu par la transaction), un **graphe de précédence** (ou de **sérialisation**) peut être produit pour tester la sérialisation des conflits. Pour une planification P , un graphe de précédence est un graphe dirigé $G = (N, F)$ constitué d'un ensemble N de nœuds et d'un ensemble F de flèches, construit comme suit :

Test de conflits de la capacité de sérialisation

- Créer un noeud pour chaque transaction.
- Créer une flèche dirigée $T_i \rightarrow T_j$, si T_j lit la valeur d'un élément écrit par T_i .
- Créer une flèche dirigée $T_i \rightarrow T_j$, si T_j écrit une valeur dans un élément après qu'il a été lu par T_i .
- Créer une flèche dirigée $T_i \rightarrow T_j$, si T_j écrit une valeur dans élément après qu'il a été écrit par T_i .
- Si une flèche $T_i \rightarrow T_j$ existe dans le graphe de précédence pour P, alors dans toute planification sérielle S' équivalente à S , T_i doit apparaître avant T_j .
- Si le graphe de précédence contient un cycle, la planification n'est pas sérialisable en vue de résoudre les conflits.

Exemple -

Deux transactions de modification concurrentes qui n'ont pas la capacité de sérialisation des conflits.



Temps	T_9	T_{10}
t_1	début_transaction	
t_2	lire(solde _x)	
t_3	solde _x = solde _x + 100	
t_4	écrire(solde _x)	début_transaction
t_5		lire(solde _x)
t_6		solde _x = solde _x * 1.1
t_7		écrire(solde _x)
t_8		lire(solde _y)
t_9		solde _y = solde _y * 1.1
t_{10}		écrire(solde _y)
t_{11}	lire(solde _y)	validation
t_{12}	solde _y = solde _y - 100	
t_{13}	écrire(solde _y)	
t_{14}	validation	

Graphe de précédence correspondant à la figure ci dessus. Le graphe présente un cycle, de sorte que la planification ne permet pas la sérialisation des conflits.

Récupération

La capacité de sérialisation identifie des planifications qui conservent toute sa cohérence à une base de données, en admettant qu'aucune transaction planifiée n'échoue.

Si une transaction défaille, la propriété d'atomicité impose que nous annulions les effets de la transaction.

En outre, la propriété de durabilité indique qu'une fois qu'une transaction est validée, il n'est plus possible d'annuler les modifications qu'elle a apportées (du moins sans exécuter une transaction distincte, de compensation).

En d'autres termes, cette planification est une planification irrécupérable qu'il ne faut pas permettre d'utiliser.

Techniques de contrôle de concurrence

L'aspect sérialisable s'obtient de diverses manières. Il existe deux techniques de contrôle de concurrence principales qui permettent d'exécuter des transactions en parallèle en toute sécurité, à condition de faire appel à certaines contraintes : les méthodes de verrouillage ou "Locking", et d'estampillage ou "Timestamping" .

Techniques de contrôle de concurrence

Le verrouillage et l'estampillage sont des approches fondamentalement **conservatrices** (ou **pessimistes**), en ceci qu'elles provoquent un retardement des transactions au cas où elles entreraient en conflit avec d'autres transactions dans un certain délai à venir.

Les méthodes **optimistes**, reposent sur l'hypothèse que les conflits sont rares, de sorte qu'elles permettent aux transactions de procéder de manière désynchronisée et ne vérifient la présence de conflits qu'en fin de transactions, lors de leur validation.

Méthodes de verrouillage

Verrouillage

Une procédure employée pour contrôler les accès concurrents aux données. Lorsqu'une transaction accède à une base de données, un verrou est susceptible de bloquer l'accès à d'autres transactions pour éviter de faux résultats.

Verrou partagé

Si une transaction dispose d'un verrou partagé sur une donnée, elle peut lire la donnée mais pas la modifier.

Verrou exclusif

Si une transaction possède un verrou exclusif sur une donnée, elle peut lire et modifier cette donnée.

Règles d'utilisation des verrous

- Toute transaction devant accéder à une donnée verrouille d'abord la donnée, demandant soit un verrouillage partagé dans le cas d'un accès en lecture, soit un verrouillage exclusif dans le cas d'un accès tant en lecture qu'en modification.
- Si la donnée n'est pas déjà verrouillée par une autre transaction, le verrou est accordé.
- Si la donnée est déjà verrouillée au moment de la demande, le SGBD détermine si la demande est compatible avec le verrou actuel. Si c'est un verrou partagé que la transaction demande, alors qu'un verrou partagé est déjà placé sur la donnée, la requête peut être satisfaite et le verrou est accordé ; dans le cas contraire, la transaction demanderesse doit **attendre** que le verrou se libère.

Règles d'utilisation des verrous

- Une transaction qui détient un verrou le conserve tant qu'elle ne le libère pas explicitement pendant l'exécution ou implicitement lorsqu'elle se termine (par une annulation ou une validation). Ce n'est que lorsqu'un verrou exclusif est libéré que les effets de l'opération d'écriture qui a motivé le verrou deviennent visibles aux autres transactions

Verrouillage en deux phases (V2P)

Une transaction suit le protocole de verrouillage en deux phases si toutes les opérations de verrouillage précèdent la première opération de déverrouillage dans la transaction.

Selon les règles de ce protocole, toute transaction est divisible en deux phases : une première phase dite de **croissance** où elle acquiert tous les verrous mais ne peut en libérer aucun, et une phase de **résorption**, au cours de laquelle elle libère tous les verrous et ne peut plus en obtenir aucun.

Verrouillage en deux phases (V2P)

Les règles sont les suivantes :

1. Une transaction doit acquérir un verrou sur un élément avant de faire quoi que ce soit sur celui-ci. Le verrou est en lecture ou en écriture, selon le type d'accès nécessaire.
 2. Une fois que la transaction libère un verrou, elle ne peut plus demander d'autres verrous.
- Si l'élévation des verrous est possible, elle ne peut avoir lieu que pendant la phase de croissance, ce qui peut l'obliger à attendre qu'une autre transaction libère un verrou partagé sur un élément. La réduction ne peut se produire que pendant la phase de résorption.

Blocage indéfini (deadlock)

L'impasse générée par deux transactions (ou plus) qui attendent, l'une, que des verrous se libèrent, alors qu'ils sont détenus par l'autre.

Temps	T ₁₇	T ₁₈
t ₁	début_transaction	
t ₂	verrou_écriture(solde _x)	début_transaction
t ₃	lire(solde _x)	verrou_écriture(solde _y)
t ₄	solde _x = solde _x - 10	lire(solde _y)
t ₅	écrire(solde _x)	solde _y = solde _y + 100
t ₆	verrou_écriture(solde _y)	écrire(solde _y)
t ₇	ATTENTE	verrou_écriture(solde _x)
t ₈	ATTENTE	ATTENTE
t ₉	ATTENTE	ATTENTE
t ₁₀	⋮	ATTENTE
t ₁₁	⋮	⋮

Blocage indéfini (deadlock)

Trois techniques permettent de gérer les verrous indéfinis:

1. imposer un délai imparti,
2. prévenir les verrous indéfinis ou les
3. détecter et les récupérer.

- Le principe du **délai imparti** est qu'une transaction est mise en attente pour une période d'une durée maximale.
- Par la **prévention des verrous indéfinis**, le SGBD vérifie, avant de l'exécuter, si une transaction est susceptible de provoquer un verrou indéfini et ne permet jamais à des verrous indéfinis de se produire.

Blocage indéfini (deadlock)

- Par la **détection et la récupération des verrous indéfinis**, le SGBD autorise les verrous indéfinis mais les reconnaît et les brise.
- Comme il est plus difficile de prévenir les verrous indéfinis que de les détecter puis de les briser ou de faire appel à des délais maximum d'attente, les systèmes évitent la méthode de la prévention des verrous indéfinis.

Méthodes d'estampillage

- Une autre approche, qui garantit aussi la sérialisation, fait appel à des estampilles de temps (c'est-à-dire des empreintes temporelles), cachetées sur les transactions, de façon à classer l'exécution des transactions dans une planification sérielle équivalente.
- Les méthodes d'estampillage dans le contexte du contrôle de concurrence diffèrent assez des méthodes de verrouillage.
- Comme elles ne font intervenir aucun verrou, elles ne peuvent donner lieu à des verrous indéfinis.
- Avec ces méthodes, il n'y a aucune attente: les transactions qui entrent en conflit sont simplement annulées et redémarrées.

Méthodes d'estampillage

Estampille

Un identificateur unique créé par le SGBD indiquant l'heure de démarrage relative d'une transaction.

Estampillage

Un protocole de contrôle de concurrence qui classe les transactions dans un ordre tel que les transactions plus anciennes, qui portent les estampilles les plus *petites*, obtiennent la plus grande priorité dans l'éventualité d'un conflit.

Tri des estampilles multiversions

- Le protocole élémentaire de classement par ordre d'estampilles évoqué à la section précédente suppose qu'une seule version existe d'une donnée et que, de ce fait, une seule transaction peut accéder à une donnée à un moment donné.
- Cette restriction peut être assouplie si nous permettons à plusieurs transactions de lire et écrire des versions différentes de la même donnée et garantissons que chaque transaction voit un ensemble cohérent de toutes les données auxquelles elle accède.
- Dans le contexte du contrôle de concurrence multiversion, toute opération d'écriture crée une nouvelle version d'une donnée au lieu de retenir l'ancienne version. Quand une transaction tente de lire une donnée, le système sélectionne la version, parmi celles existantes, qui assure la sérialisation.

Techniques optimistes

- Dans certains environnements, les conflits parmi les transactions sont rares et le traitement nécessaire aux protocoles de verrouillage ou d'estampillage est inutile pour nombre de transactions.
- Les **techniques optimistes** se basent sur l'hypothèse de la rareté des conflits et qu'il est plus efficace de permettre aux transactions de s'exécuter sans leur imposer de délai en relation avec la sérialisation.
- Un protocole de concurrence optimiste comporte généralement de deux à trois phases, selon qu'il s'adresse à une transaction de lecture seule ou de mise à jour :

Techniques optimistes

- La **phase de lecture** s'étale du début de la transaction jusque avant la validation (commit). La transaction lit les valeurs de toutes les données dont elle a besoin dans la base de données et les stocke dans des variables locales. Les mises à jours sont appliquées aux copies locales des données et non à la base de données.
- La **phase de validation** suit immédiatement la phase de lecture. Des contrôles y sont menés pour vérifier que la sérialisation n'est pas transgressée si les mises à jour par la transaction sont appliquées à la base de données. Dans le cas d'une transaction en lecture seule, ceci suppose des vérifications que les valeurs des données lues sont toujours les valeurs des données correspondantes. Si aucune interférence ne se manifeste, la transaction est validée.

Techniques optimistes

- La ***phase de validation*** .
- Si des interférences apparaissent, la transaction est annulée et redémarrée. Dans le cas d'une transaction avec mises à jour, la validation détermine que la transaction courante laisse la base de données dans un état cohérent, dont la sérialisation est conservée. Si tel n'est le cas, la transaction est annulée et redémarrée.
- La ***phase d'écriture*** suit une phase de validation réussie dans le cas des transactions en écriture. Au cours de cette phase, les modifications apportées à la copie locale sont appliquées à la base de données.

Granularité des données

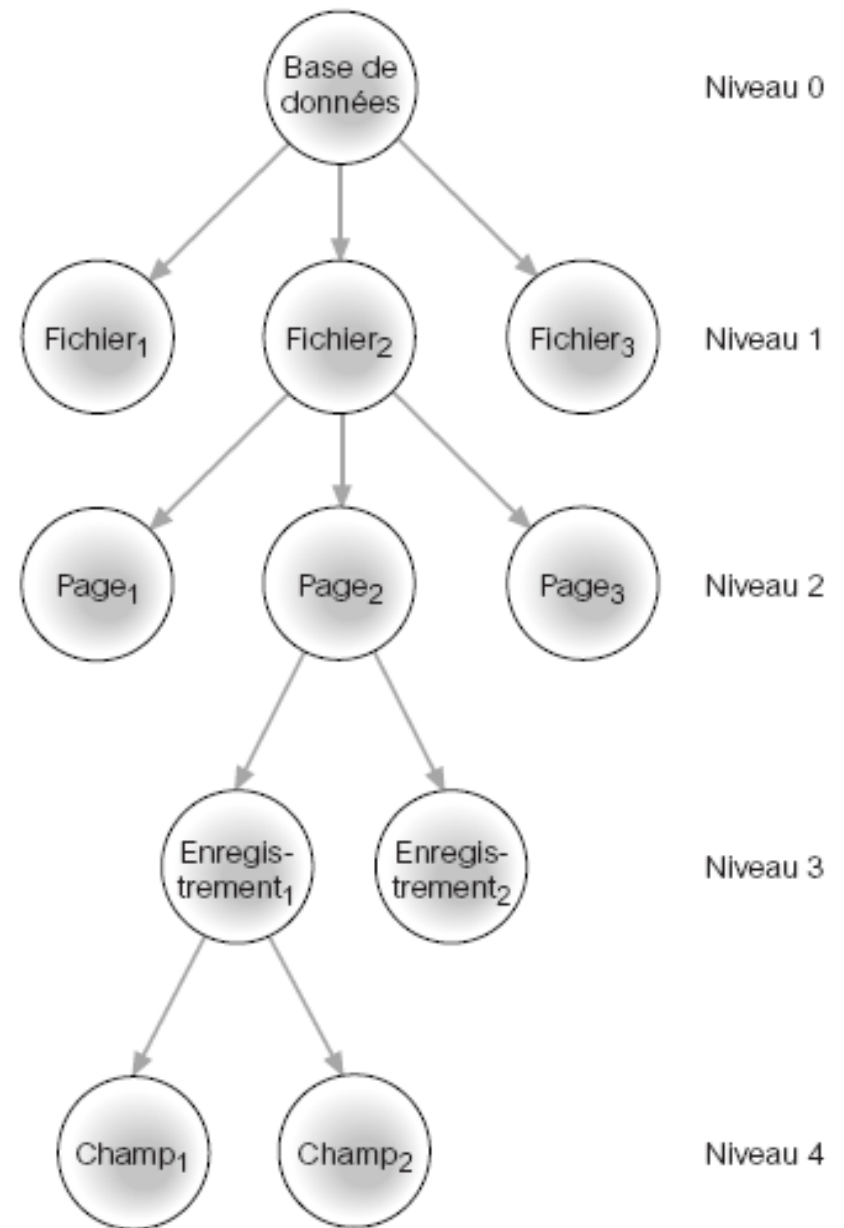
- **Granularité** La taille des données choisies comme l'unité de protection par un protocole de contrôle de concurrence.
- Tous les protocoles de contrôle de concurrence que nous avons évoqués supposent que la base de données est formée d'un certain nombre de « données », sans définir explicitement le terme.
- D'une façon type, une donnée est considérée comme un des éléments suivants, du plus vaste au plus étroit, où une granularité fine fait référence à un élément de petite taille et une granularité vaste à des éléments de taille importante:
 1. La base de données dans sa totalité;
 2. Un fichier;
 3. Une page (appelée parfois une zone ou un espace de base de données, c'est-à-dire une
 4. section du disque physique dans laquelle des relations sont inscrites);
 5. Un enregistrement;
 6. Une valeur de champ d'un enregistrement..

Hiérarchie de granularité

Nous pourrions représenter la granularité des verrous sous la forme d'une structure hiérarchique dont chaque noeud représente des données de tailles différentes.

- Le noeud racine représente la base de données dans sa totalité,
- les noeuds de niveau 1 représentent les fichiers,
- les noeuds de niveau 2 représentent des pages,
- les noeuds de niveau 3 représentent des enregistrements et
- les feuilles de niveau 4 représentent des champs individuels.

Hiérarchie de granularité



Hiérarchie de granularité

- Chaque fois qu'un noeud est verrouillé, tous ses descendants le sont également. Si une autre transaction demande un verrou incompatible sur le *même* noeud, le SGBD sait sans ambiguïté qu'il ne peut accorder le verrou.
- Le SGBD contrôle le chemin hiérarchique depuis la racine jusqu'au noeud demandé, pour déterminer si un ancêtre du sujet de la demande est déjà verrouillé, avant d'accepter la concession du verrou.

Un verrou d'intention peut s'utiliser pour verrouiller tous les ancêtres d'un nœud verrouillé.

Un verrou d'intention peut être en lecture ou en écriture, appliqué de haut en bas, relâché de bas en haut

Hiérarchie de granularité

- Aucun verrou ne peut être accordé dès qu'un noeud vient d'être déverrouillé.
- Aucun noeud ne peut être verrouillé, tant que son parent est verrouillé par un verrou d'intention.
- Un noeud ne peut être déverrouillé que lorsque tous ses descendants sont déverrouillés.
- De cette manière, les verrous s'appliquent du noeud racine jusqu'aux noeuds feuilles à l'aide de verrous d'intention jusqu'à atteindre le noeud qui nécessite un véritable verrou en lecture ou exclusif, et les verrous sont libérés des feuilles à la racine. Un verrou indéfini demeure néanmoins toujours possible, auquel cas, il doit être géré comme indiqué plus haut.

Tableau 20.1 Tableau de compatibilité du verrouillage à granularités multiples.

	PI	XI	P	PXI	X
<i>PI</i>	✓	✓	✓	✓	✗
<i>XI</i>	✓	✓	✗	✗	✗
<i>P</i>	✓	✗	✓	✗	✗
<i>PXI</i>	✓	✗	✗	✗	✗
<i>X</i>	✗	✗	✗	✗	✗

✓ = compatible, ✗ = incompatible

Récupération d'un BD

Types de pannes

- Les **plantages système** dus aux erreurs de matériel ou de logiciels, conduisant à la perte de la mémoire principale.
- Les **défaillances de supports de données**, telles que les cassures de tête de lecture ou les supports illisibles entraînant la perte de parties du stockage secondaire.
- Les **erreurs de logiciel d'application**, telles que les erreurs logiques dans un programme qui accède à la base de données, provoquant la défaillance d'une ou plusieurs transactions.

Types de pannes

- Les **catastrophes naturelles**, comme les incendies, les inondations, les tremblements de terre ou les coupures d'alimentation électrique.
- Le **manque de soin** ou la destruction non intentionnelle des données ou des utilitaires par les opérateurs ou les utilisateurs.
- Le **sabotage**, c'est-à-dire la corruption ou la destruction intentionnelle des données, du matériel ou des utilitaires logiciels.

Types de pannes

- Quelle que soit la cause de la défaillance, nous devons tenir compte de deux de ses effets :
- la perte de la mémoire principale, y compris des tampons de base de données, et
- la perte de la copie sur disque de la base de données.

Transactions et récupération

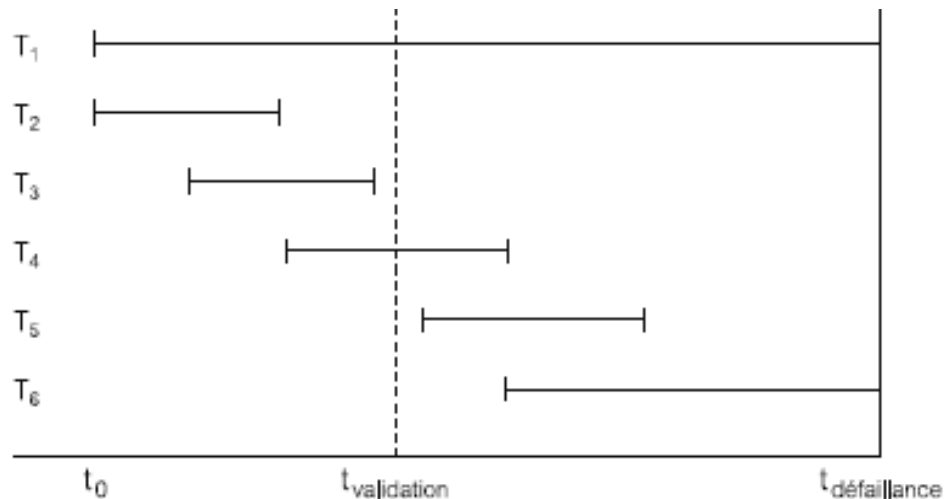
- Dans un système de base de données, les transactions représentent *l'unité de récupération*.
- Le gestionnaire de récupération est chargé de garantir deux des quatre propriétés ACID des transactions, soit l'atomicité et la durabilité, en présence de défaillances.
- Le gestionnaire de récupération doit s'assurer de ce que, lors de la récupération après une défaillance, soit *tous* les effets d'une transaction donnée sont enregistrés de manière permanente dans la base de données, soit *aucun* d'entre eux.

Transactions et récupération

- La situation se complique du fait que l'écriture dans une base de données n'est pas une action atomique (en une seule étape) et qu'il se peut donc qu'une transaction ait été validée, mais que ses effets n'aient pas encore été enregistrés dans la base de données, simplement parce qu'ils n'ont pas encore pu atteindre la base de données.

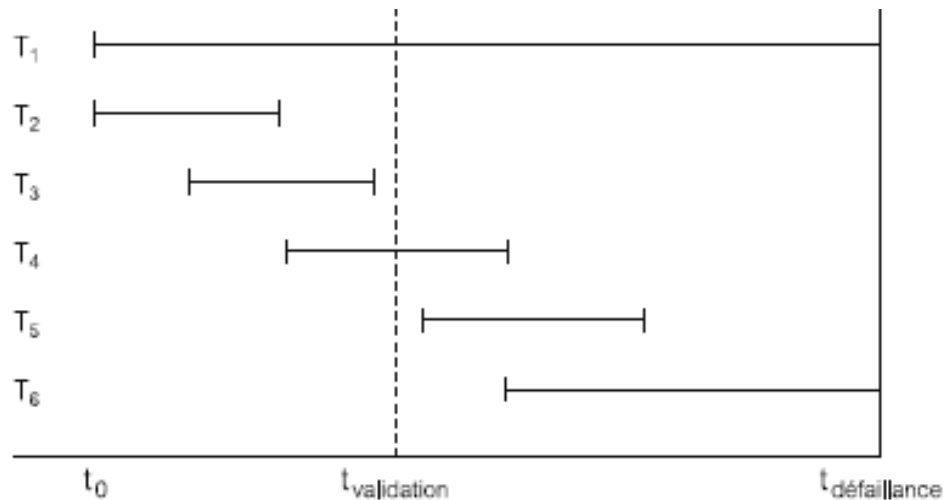
Exemple :

- Les transactions, $T_1, \dots, T_6$ qui s'exécutent simultanément.
- Si le SGBD démarre au temps t_0 et défaille au temps t_d . Supposons que les données des transactions T_2 et T_3 ont été écrites dans le stockage secondaire avant la défaillance.



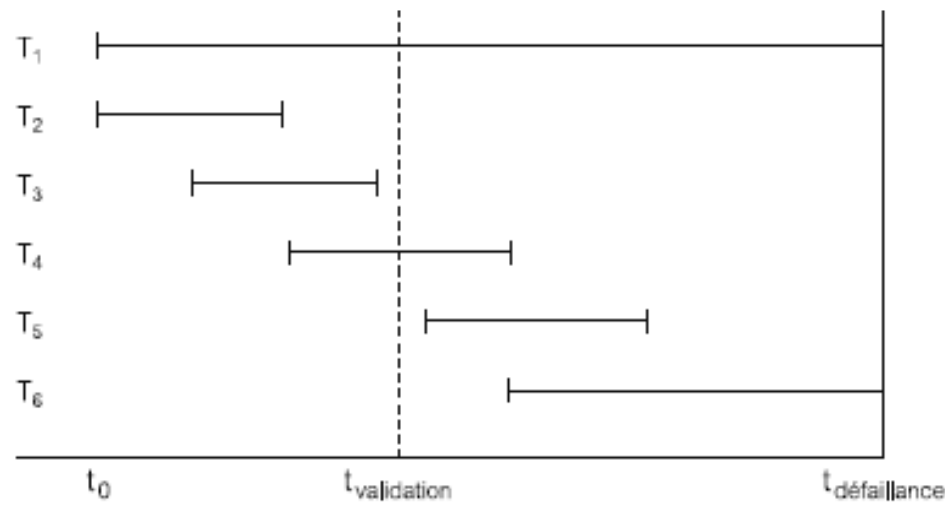
Exemple :

- En clair, T1 et T6 ne se sont pas validées au moment du plantage ; de ce fait, au redémarrage, le gestionnaire de récupération doit défaire (*undo*) les transactions T1 et T6. Cependant, il n'est pas évident de savoir jusqu'à quel point les modifications apportées par les autres transactions



Exemple :

- T4 et T5 validées ont été propagées dans la base de données sur le stockage non volatile. La raison de cette incertitude réside dans le fait que les tampons de base de données volatiles ont peut-être été écrits sur disque ou peut-être pas.
- En l'absence de toute autre information, le gestionnaire de récupération doit imposer le *redo* des transactions T2, T3, T4 et T5.



Gestion des tampons

- La gestion des tampons de base de données joue un rôle important dans le processus de récupération;
- Il est responsable de l'administration efficace des tampons de base de données utilisés pour transférer des pages vers et en provenance du stockage secondaire.
- Une **stratégie de vol** permet au gestionnaire de tampons d'écrire un tampon sur le disque avant qu'une transaction ne soit validée (le tampon est déconsigné).
- En d'autres termes, le gestionnaire de tampons « dérobe » une page à la transaction.

Gestion des tampons

- La stratégie opposée est dite **antivol** :
- Une **stratégie de forçage** s'assure de ce que toutes les pages modifiées par une transaction sont écrites sur disque quand la transaction se valide.
- La stratégie alternative est l'**antiforçage**.

Utilitaires de récupération

Un SGBD doit proposer les utilitaires suivants, destinés à faciliter la récupération :

- Un mécanisme de sauvegarde, qui s'occupe des sauvegardes périodiques de copies de la base de données.
- Des outils de journalisation, qui conservent une trace de l'état actuel des transactions et des modifications de la base de données.
- Un utilitaire de points de contrôle, qui permet de rendre permanentes des modifications de la base de données en cours de progression.
- Un gestionnaire de récupération, qui permet de restaurer la base de données dans un état cohérent à la suite d'une défaillance.

Fichier de journalisation

- Pour garder une trace de toutes les transactions de la base de données, le SGBD entretient un fichier spécial, appelé un **journal** (ou **log**) contenant les informations des mises à jour appliquées à la base de données. Le journal comporte les données suivantes :
 - Les **enregistrements de transactions**, contenant :
 - – L'identificateur de transaction ;
 - – Le type d'enregistrement de journal (début de transaction, ajout, modification, suppression, annulation, validation) ;

Fichier de journalisation

- **L'image avant de la donnée**, c'est-à-dire sa valeur avant le changement (seulement pour la modification et la suppression) ;
- **L'image après de la donnée**, c'est-à-dire sa valeur après le changement (seulement pour les opérations d'ajout et de modification) ;
- Les informations de gestion du journal, comme un pointeur vers les enregistrements de journal précédent et suivant, pour cette transaction (toutes opérations).
- Les enregistrements des points de contrôle

Points de contrôle

- Le point de synchronisation entre la base de données et le fichier journal de transaction. L'écriture de tous les tampons est alors forcée sur le stockage secondaire.,

Modèles avancés de transaction

- **Modèle de transactions imbriquées**

- Une transaction est vue comme une collection de sous-tâches correspondantes ou *sous-transactions*, qui peuvent chacune contenir aussi un nombre quelconque de sous-transactions.

- **Saga** Une séquence de transactions (à plat) qu'il est possible d'interfolier avec d'autres transactions.

- **Modèle de transactions à plusieurs niveaux**

- **Restructuration dynamique**

- **Modèles de flux d'activité (*workflow*)**

Cf. A. G.

Références :

- Livres de A.Gamache
- Support de M.P.Parent